



觉得还不错? 一键收藏



FREE_QIU

关注



16



80



Python Flask - 实现本地文件的上传与下载

原创

FREE_QIU

已于 2022-11-17 13:43:32 修改

阅读量7.2k

收藏 80

点赞数 16

分类专栏: Flask Python

文章标签: flask python 后端

 华为云开发者联盟 该内容已被华为云开发者联盟社区收录

 Flask 同时被 2 个专栏收录

2 订阅 4 篇文章

场景：

- 点击上传文件按钮，选择需要上传的文件后上传
 - 文件上传成功后，会将文件保存到指定目录下
 - 限制上传文件的格式
- 在前端点击文件后下载
 - 基于上述上传并保存到指定目录下的文件

上手

文件上传

app.py

```
1
2 @app.route('/upload', methods=['GET', 'POST'])
3 def upload_file():
4     # 渲染文件
5     return render_template('upload.html')
6
7 # 文件保存的目录，根据实际情况的文件结构做调整；
8 # 若不指定目录，可以写成f.save(f.filename)，可以默认保存到当前文件夹下的根目录
9 # 设置上传文件保存路径 可以是指定绝对路径，也可以是相对路径（测试过）
10 app.config['UPLOAD_FOLDER'] = './upload'
11 # 将地址赋值给变量
12 file_dir = app.config['UPLOAD_FOLDER']
13
14
15 @app.route('/uploader', methods=['GET', 'POST'])
16 def uploader():
17     """
18     文件上传
19     """
20     if request.method == 'POST':
21         # input标签中的name的属性值
22         f = request.files['file']
23
24         # 拼接地址，上传地址，f.filename：直接获取文件名
25         f.save(os.path.join(app.config['UPLOAD_FOLDER'], f.filename))
26         # 输出上传的文件名
27         print(request.files, f.filename)
28
29         return '文件上传成功!'
30     else:
31         return render_template('upload.html')
```

upload.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3     <head>
4         <meta charset="UTF-8">
5         <title>上传</title>
```

觉得还不错？ 一键收藏



FREE_QIU

关注



16



80



```

6
7     <style>
8         a{
9             text-decoration: none;
10            color: #2062e0;
11        }
12        a:hover{
13            text-decoration: none;
14            color: #2062e0;
15        }
16    </style>
17 </head>
18 <body>
19     <h1 align="center">TEST</h1>
20     <div align="center">
21         {# enctype: 属性用于指定将上传文件数据发送给服务器端之前对表单数据进行编码的格式 #}
22         {# enctype="multipart/form-data" => 表示不对字符编码。当使用有文件上传控件的表单时，该值是必需的。 #}
23         <form action="/uploader" method="post" enctype="multipart/form-data">
24             <br><br><br>
25             {# accept可以自定义文件上传格式 #}
26             <input type="file" name="file" accept=".txt, .pdf, .doc, .docx, .md" value="{{ csrf_token }}" />
27             <br><br><br>
28             <input type="submit" value="提交" />
29         </form>
30
31         <br><br><br>
32     </div>
33 </body>
34 </html>

```

文件下载

app.py

```

1  @app.route('/download', methods=['GET', 'POST'])
2  def download():
3      """
4          文件下载
5      :return:
6      """
7      timelist = [] # 获取指定文件夹下文件并显示
8      Foder_Name = [] # 文件夹下所有文件
9      Files_Name = [] # 文件名
10
11     # 获取到指定文件夹下所有文件
12     lists = os.listdir(file_dir + '/')
13
14     # 遍历文件夹下所有文件
15     for i in lists:
16         # os.path.getatime => 获取对指定路径的最后访问时间
17         timelist.append(time.ctime(os.path.getatime(file_dir + '/' + i)))
18
19     # 遍历文件夹下的所有文件
20     for k in range(len(lists)):
21         # 单显示文件名
22         Files_Name.append(lists[k])
23         # 获取文件名以及时间信息
24         Foder_Name.append(lists[k] + " ~~~~~~ " + timelist[k])
25
26     print(file_dir) # ./upload
27
28     return render_template('download.html', allname=Foder_Name, name=Files_Name)
29
30
31 @app.route('/downloads/<path:path>', methods=['GET', 'POST'])
32 def downloads(path):
33     """
34         重写download方法，根据前端点击的文件传送过来的path，下载文件
35
36         send_from_directory: 用于下载文件
37         flask.send_from_directory(所有

```

觉得还不错? 一键收藏

 16
 
 80
 

```

38     :param path:
39     :return:
40     """
41     return send_from_directory(app.config['UPLOAD_FOLDER'], path, as_attachment=True)
42

```

download.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title>文件下载</title>
6
7          <style>
8              a{
9                  text-decoration: none;
10                 color: #2062e0;
11             }
12             a:hover{
13                 text-decoration: none;
14                 color: #2062e0;
15             }
16         </style>
17
18     </head>
19     <body>
20         <div align="center">
21             <h1>文件下载</h1><br><br>
22             {# 输出文件名及文件详细信息(文件时间信息等) #}
23             {% for fruit in allname %}
24                 <br>
25                 {{ fruit }}
26             {% endfor %}
27             <br><br><br><br>
28             {# 将指定文件夹中的文件获取遍历显示 #}
29             {% for n in name %}
30
31                 <a href="downloads/{{ n }}">{{ n }}</a>
32                 <br><br>
33
34             {% endfor %}
35         </div>
36     </body>
37 </html>

```

运行

- 文件上传和下载的视图函数代码都完成后，开始运行项目

app.py (在编写好试图代码的最下方编写运行代码)

```

1  """
2      运行项目
3  """
4  if __name__ == '__main__':
5      # 可以使统一局域网下的其他电脑访问该项目
6      HOST = '0.0.0.0'
7      # debug=True => 打开调试模式, 在对代码进行修改后, 可以实时运行代码
8      app.run(host=HOST, debug=True)

```

- 打开调试模式后，运行项目建议在Pycharm中的Terminal命令行中输入以下运行

```
1 | python app.py
```

觉得还不错? 一键收藏

整个app.py代码



FREE_QIU

关注

16



80



```
1 import os
2 import time
3
4 from flask import Flask, render_template, request, send_from_directory
5
6 app = Flask(__name__)
7
8
9 @app.route('/')
10 def hello_world():
11     return 'Hello World!'
12
13
14 @app.route('/upload', methods=['GET', 'POST'])
15 def upload_file():
16     # 渲染文件
17     return render_template('upload.html')
18
19
20 # 文件保存的目录, 根据实际情况的文件结构做调整;
21 # 若不指定目录, 可以写成f.save(f.filename), 可以默认保存当前文件夹下的根目录
22 # 设置上传文件保存路径 可以是指定绝对路径, 也可以是相对路径 (测试过)
23 app.config['UPLOAD_FOLDER'] = './upload'    ## 该目录需要自行创建
24 # 将地址赋值给变量
25 file_dir = app.config['UPLOAD_FOLDER']
26
27
28 @app.route('/uploader', methods=['GET', 'POST'])
29 def uploader():
30     """ 文件上传 """
31     if request.method == 'POST':
32         # input标签中的name的属性值
33         f = request.files['file']
34
35         # 拼接地址, 上传地址, f.filename: 直接获取文件名
36         f.save(os.path.join(app.config['UPLOAD_FOLDER'], f.filename))
37         # 输出上传的文件名
38         print(request.files, f.filename)
39
40         return '文件上传成功!'
41     else:
42         return render_template('upload.html')
43
44
45 @app.route('/download', methods=['GET', 'POST'])
46 def download():
47     """ 文件下载 """
48     timelist = [] # 获取指定文件夹下文件并显示
49     Foder_Name = [] # 文件夹下所有文件
50     Files_Name = [] # 文件名
51
52     # 获取到指定文件夹下所有文件
53     lists = os.listdir(file_dir + '/')
54
55     # 遍历文件夹下所有文件
56     for i in lists:
57         # os.path.getatime => 获取对指定路径的最后访问时间
58         timelist.append(time.ctime(os.path.getatime(file_dir + '/' + i)))
59
60     # 遍历文件夹下的所有文件
61     for k in range(len(lists)):
62         # 单显示文件名
63         Files_Name.append(lists[k])
64         # 获取文件名以及时间信息
65         Foder_Name.append(lists[k] + " ~~~~~~ " + timelist[k])
66
67     print(file_dir) # ./upload
68
69     return render_template('download.html')
70
71
```

觉得还不错? 一键收藏



FREE_QIU

关注



16



80



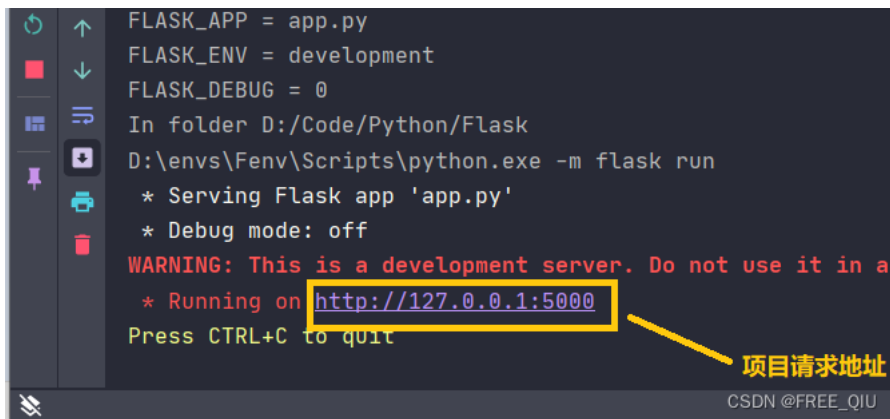
```
72 @app.route('/downloads/<path:path>', methods=['GET', 'POST'])
73 def downloads(path):
74     """ 下载 """
75     """
76     重写download方法，根据前端点击的文件传送过来的path，下载文件
77     send_from_directory: 用于下载文件
78     flask.send_from_directory(所有文件的存储目录，相对于要下载的目录的文件名，as_attachment: 设置为True是否要发送带有标题的文件)
79     """
80     return send_from_directory(app.config['UPLOAD_FOLDER'], path, as_attachment=True)
81
82
83 """
84 运行项目
85 """
86 if __name__ == '__main__':
87     # 可以使统一局域网下的其他电脑访问该项目
88     HOST = '0.0.0.0'
89     # debug=True => 打开调试模式，在对代码进行修改后，可以实时运行代码
90     app.run(host=HOST, debug=True)
```

模板文件中代码已完整

注意

- 代码编写好后，需要在项目根目录先创建一个路径用于存放上传的文件
 - 即创建upload文件夹
- 运行项目后，需要在浏览器地址后补充输入 /upload，即可进入上传页面
 - 即 <http://127.0.0.1:5000/upload>，具体端口号需根据个人项目运行修改，可以在运行时看控制台

127.0.0.1:5000/upload CSDN @FREE_QIU



```
FLASK_APP = app.py
FLASK_ENV = development
FLASK_DEBUG = 0
In folder D:/Code/Python/Flask
D:\envs\Fenv\Scripts\python.exe -m flask run
* Serving Flask app 'app.py'
* Debug mode: off
WARNING: This is a development server. Do not use it in a
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

项目请求地址

- 下载页面

127.0.0.1:5000/download CSDN @FREE_QIU

项目文件目录

觉得还不错? 一键收藏



FREE_QIU

关注

16

80

¥

6/10

kismet_web_viewer:将Kismet netxml文件解析为可读表。使用python Flask
Kismet Web查看器 将netxml文件解析为可读表。 Kismet网络查看器使用： 数据表 引导程序 烧瓶 json2html 关于Kismet Web查看器 该项目旨在在本地主机系统上为受信

flask-rest:基于Flask Python框架的RESTful服务
基于Flask Python框架的RESTful服务 Docker容器 此设置需要Docker在您的PC上运行。 我已经使用Windows和Powershell运行命令。 您可以在相应的链接中找到适用于

flask-s3-save-example:Flask应用程序示例，该应用程序使用s3-saver，URL-for-s3，flask-thumbnails-s3和flask-admin-s3-upload在Amazon S3上存
Flask应用程序示例，该应用程序使用， ， 和在Amazon S3上存储和检索文件。 此应用可让您上传，重新上传和删除单个图像。 通过更改几行配置，图像将存储在本地

keras-flask-deployment
将Keras模型上传到Flask服务器 几乎所有代码都从进行了修改。变更日志可以加载和使用aws s3中保存的模型。 将argparse添加到index.py-端口：指定端口（默认= 8080

python学习笔记——flask之上传下载文件 小橙子的
request.files['pic']中的pic是属性不是文件名 文件上传 一、普通方式 1.request中获取文件对象 2.读取数据，再写入到服务器文件中。 # coding:utf-8 from flask import Flask

flask-python搭建博客网站
Flask是一个基于Python的轻量级Web框架，通过使用它我们可以快速搭建起一个个性化的博客网站。 首先，我们需要安装Flask。可以使用pip命令来进行安装，如下所示

“相关推荐”对你有帮助么？

非常没帮助 没帮助 一般 有帮助 非常有帮助

关于我们 招贤纳士 商务合作 寻求报道 400-660-0108 kefu@csdn.net 在线客服 工作时间 8:30-22:00
公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 经营性网站备案信息 北京互联网违法和不良信息举报中心
家长监护 网络110报警服务 中国互联网举报中心 Chrome商店下载 账号管理规范 版权与免责声明 版权申诉 出版物许可证 营业执照
©1999-2023北京创新乐知网络技术有限公司

FREE_QIU

码龄4年 暂无认证

39

11万+

3万+

4万+

原创

周排名

总排名

访问

等级

456

7

38

11

270

积分

粉丝

获赞

评论

收藏

私信

关注

参与话题写文章得原力分，点亮勋章

去发布

搜博主文章

热门文章

Python Flask - 实现本地文件的上传与下载
7221
Python —— 将两个列表组成一个字典
5488

觉得还不错？ 一键收藏

FREE_QIU 关注

16 80

Jupyter Notebook——查看当前文件保存位置 3184

IDEA 配置tomcat + 配置时报错Warning: No artifacts configurae解决 + 运行tomcat后报错not found for the web module 3130

Python——降低pip的版本 3084

分类专栏

Python习题

1篇

Nginx

9篇

IDEA

9篇

Java

19篇

Python

1篇

编辑器

1篇

最新评论

Python Flask - 实现本地文件的上传与下载
2301_80438782: 大哥 怎么改

Python__关于命令出错:(-2147352567, '...
FREE_QIU: 不管是绝对路径还是相对路径, 路径中涉及到文件夹要提前创建

Python__关于命令出错:(-2147352567, '...
赵鸣漩: 绝对路径?

Python__关于命令出错:(-2147352567, '...
赵鸣漩: 这是啥意思

Python Flask - 实现本地文件的上传与下载
深世易诗: 谢谢!!

您愿意向朋友推荐“博客详情页”吗?

强烈不推荐

不推荐

一般般

推荐

强烈推荐

最新文章

PTA_编程题_1001 (3n+1)猜想

Python_关于使用【zipfile】解压路径中文乱码问题

Pycharm通用设置&个性化设置

2023年 8篇

2022年 19篇

2021年 13篇

2019年 1篇

目录

场景:

上手

文件上传

文件下载

运行

整个app.py代码

FREE_QIU

关注

16

80

觉得还不错? [一键收藏](#)

https://blog.csdn.net/weixin_45681435/article/details/125681067

9/10

注意

项目文件目录

觉得还不错? [一键收藏](#)



FREE_QIU

关注



16



80

